

Documentação Técnica

Projeto Final – ETL Automatizado de Dados Financeiros

1. Visão Geral do Projeto

O objetivo deste projeto foi desenvolver um pipeline ETL (Extract, Transform, Load) automatizado para processar dados do dataset **Credit Score Classification**, disponibilizado no Kaggle.

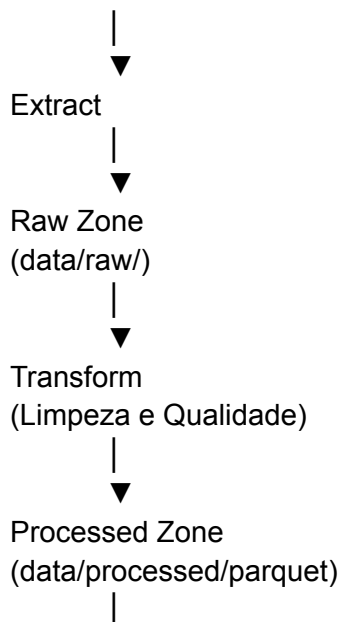
A solução foi construída para atender ao cenário de negócio da empresa fictícia **Data Girls Finance**, responsável por análises de crédito e avaliação de risco de clientes.

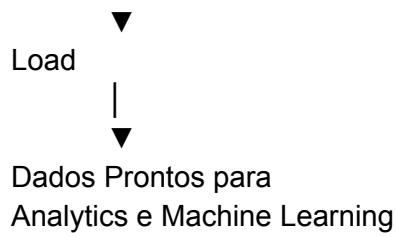
O pipeline realiza a extração dos dados, executa etapas de limpeza e validação, gera um conjunto de dados tratado em formato Parquet e disponibiliza o resultado para consumo por equipes de Analytics e Ciência de Dados.

Arquitetura da Solução

A arquitetura foi dividida em três camadas principais:

Dataset Credit Score Classification (CSV)





Estrutura do Projeto

Projeto_Final_ETL/

```
|— airflow/
| |— dags/
| |— scripts/
| |— logs/
| |— docker-compose.yaml
|
|— dags/
| |— credit_score_pipeline.py
|
|— scripts/
| |— extract.py
| |— transform.py
| |— load.py
|
|— data/
| |— raw/
| | |— train.csv
| | |— test.csv
| |
| |— processed/
| | |— credit_score/
|
|— README.md
```

Tecnologias Utilizadas

- Python
- Pandas

- Apache Airflow
 - Docker Desktop
 - Apache Parquet
 - Git
 - GitHub
-

Etapa 1 – Extração (Extract)

Objetivo

Garantir que os dados estejam disponíveis para processamento.

Implementação

Inicialmente o projeto foi planejado para utilizar a API do Kaggle. Entretanto, durante a implementação, optou-se por utilizar o arquivo CSV local disponibilizado pelo próprio dataset.

Essa abordagem foi escolhida por:

- reduzir dependências externas;
- aumentar a estabilidade do pipeline;
- evitar falhas de autenticação da API;
- simplificar a execução automatizada no Airflow.

Validação realizada

O script verifica se o arquivo existe antes de prosseguir:

`data/raw/train.csv`

Caso o arquivo não seja encontrado, o pipeline interrompe a execução e registra o erro.

Etapa 2 – Transformação (Transform)

Objetivo

Converter os dados brutos em um conjunto limpo e consistente para consumo analítico.

Principais transformações realizadas

Remoção de registros duplicados

A remoção de duplicidades evita inconsistências em análises futuras.

```
df.drop_duplicates()
```

Tratamento de valores inválidos

O dataset contém alguns campos preenchidos com caracteres especiais e valores inconsistentes.

Exemplos:

Esses valores foram convertidos para nulos.

Conversão de tipos numéricos

Diversas colunas financeiras estavam armazenadas como texto.

Exemplos:

- Annual_Income
- Monthly_Inhand_Salary
- Outstanding_Debt
- Monthly_Balance

Essas colunas foram convertidas para numéricas utilizando:

```
pd.to_numeric(errors="coerce")
```

Quando encontrado um valor inválido, ele é substituído por valor nulo.

Padronização dos dados

Foi aplicada padronização dos tipos para garantir compatibilidade com ferramentas analíticas e modelos preditivos.

Geração do formato Parquet

Após a limpeza dos dados, o resultado foi salvo em formato Parquet.

Justificativa:

- menor tamanho de armazenamento;
 - maior desempenho em leitura;
 - amplamente utilizado em Data Lakes;
 - compatível com ferramentas de Big Data.
-

Etapas 3 – Carga (Load)

Objetivo

Disponibilizar os dados processados para consumo posterior.

O resultado final é armazenado em:

`data/processed/credit_score/`

Formato `.parquet` Essa camada representa a zona tratada do pipeline.

Automação com Apache Airflow

Objetivo

Automatizar a execução das etapas ETL.

Foi criada a DAG:

`credit_score_pipeline`

Com três tarefas principais:

extract



transform



load

..

Implementação

O Airflow foi executado em ambiente Docker.

Cada etapa do pipeline é executada através de um script Python independente.

Task 1 – Extract

Responsável por localizar e validar o dataset.

Task 2 – Transform

Responsável pela limpeza, padronização e geração do arquivo Parquet.

Task 3 – Load

Responsável pela disponibilização do resultado final.

Logging e Monitoramento

O Apache Airflow foi utilizado como ferramenta de monitoramento.

Benefícios:

- rastreamento de falhas;
- histórico de execuções;

- visualização de dependências;
- reexecução de tarefas específicas.

Durante os testes foi possível identificar e corrigir problemas relacionados a:

- dependências Python;
 - montagem de volumes Docker;
 - acesso aos arquivos;
 - tipagem de colunas;
 - conversão para formato Parquet.
-

Integração com Power BI

Foi desenvolvido um dashboard utilizando Power BI conectado diretamente ao arquivo `credit_score.parquet` gerado pelo pipeline ETL.

O dashboard apresenta indicadores de negócio relevantes para análise de crédito:

- Total de clientes cadastrados;
- Distribuição dos clientes por classificação de score;
- Renda média dos clientes;
- Dívida média;
- Quantidade de empréstimos;
- Distribuição da renda mensal por categoria de score;
- Saldo médio e endividamento por classificação de crédito.

Essa integração demonstra como os dados processados pelo pipeline podem ser consumidos por ferramentas de Business Intelligence para apoiar a tomada de decisão e análises financeiras.

Respostas às Perguntas Norteadoras

1. Como garantir que os dados estejam sempre atualizados?

Os dados são atualizados através do agendamento automatizado do pipeline no Airflow.

A execução programada elimina a necessidade de intervenção manual e garante que os dados sejam processados periodicamente.

Além disso, os logs permitem monitorar falhas e reprocessar execuções quando necessário.

2. Quais validações de qualidade foram realizadas?

Foram implementadas validações como:

- verificação da existência do arquivo de entrada;
 - remoção de duplicidades;
 - tratamento de valores nulos;
 - conversão de tipos numéricos;
 - padronização dos dados;
 - validação dos campos financeiros antes da geração do Parquet.
-

3. Como evitar duplicação de registros?

A estratégia adotada foi a remoção de registros duplicados durante a etapa de transformação.

Essa abordagem garante consistência dos dados e evita contagens indevidas em análises futuras.

4. Como organizar os dados para análises futuras?

A estrutura foi organizada em camadas:

raw/

processed/

A camada RAW armazena os dados originais.

A camada PROCESSED armazena os dados limpos e prontos para consumo.

Essa organização segue boas práticas de Data Lake e facilita a integração com:

- Power BI;

- ferramentas de Analytics;
- modelos de Machine Learning;
- bancos de dados e ambientes de nuvem.

Conclusão

O projeto entregou um pipeline ETL automatizado capaz de extrair, transformar e disponibilizar dados financeiros para análise de crédito. A solução foi implementada utilizando Python, Pandas, Docker e Apache Airflow, seguindo conceitos fundamentais de Engenharia de Dados, automação de processos e qualidade dos dados.

A arquitetura construída permite evolução futura para integração com serviços de nuvem como AWS S3, Azure Blob Storage ou Google Cloud Storage, mantendo a escalabilidade e a governança dos dados.

